# Technical Documentation

Impact Circles — AI Automation Assessment Deliverable

**Organization:** Impact Circles
**Author:** Ahmed Abd EL-Kareem
**Stack:** Next.js 16 • Neon DB • Gemini AI • n8n
**Date:** October 2026

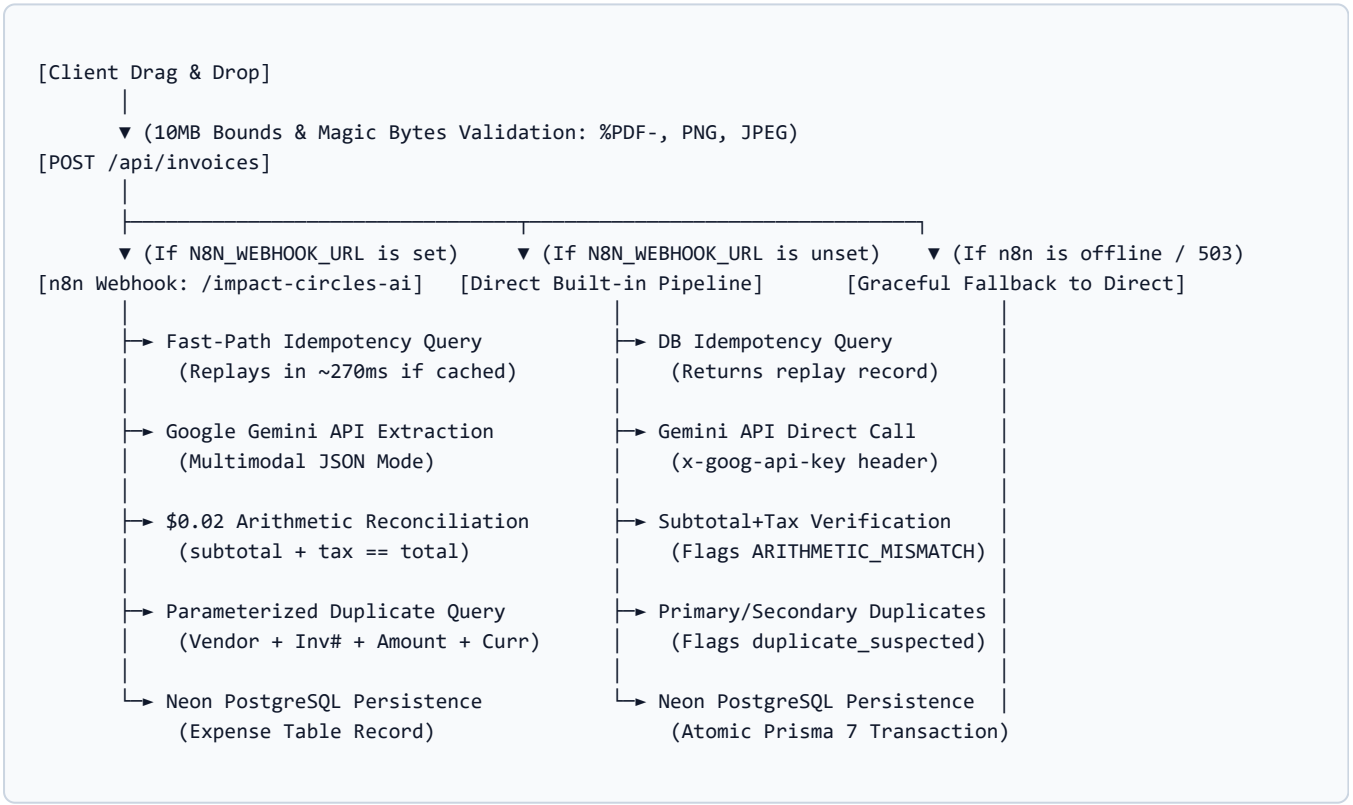
> **Document Scope & Assessment Alignment**
>
> This technical documentation fulfills Section 5 of the Technical Assessment specification. It provides a comprehensive analysis of the system architecture, underlying toolchain, production-grade integrations, mathematical/security invariants, limitations, and reproducible step-by-step instructions.

## 1. Architecture & Processing Flow

The system is engineered as two cohesive vertical slices unified under a single full-stack architecture running **Next.js 16 (App Router)**, **Neon Serverless PostgreSQL** via **Prisma 7**, **Google Gemini API** multimodal models, and optional **n8n workflow orchestration**.

### A. Financial Document Assistant (Phase 2)

Processes financial receipts and invoices across PDF, PNG, and JPEG formats through multimodal extraction, strict arithmetic/schema verification, duplicate detection, and request idempotency.




### B. AI Content Generation & Automated Publishing Agent (Phase 3)

Transforms research updates and event announcements into three channel-specific formats (LinkedIn, Newsletter, Website), provides interactive human review and editing, enforces strict approval invariants, and dispatches to multi-channel sandbox targets with full audit logging.


[Research Update Prompt] ⟶ [POST /api/content]
├⟶ Google Gemini AI (Structured JSON Mode)
├⟶ Schema Validation (Ensures usable copy on all 3 formats)
└⟶ Neon PostgreSQL (Saved as status: "draft")
[Human-in-the-Loop Review Sheet]
• 3-Format Interactive Editor (LinkedIn, Newsletter, Website)
• Channel Selection Checkboxes
• Passcode Security (x-reviewer-passcode header)
• Re-Approval Invariant: Edits reset status to "in_review"
▼ [Explicit Human Approval]
[Status: "approved", approvedAt: timestamp]
▼ [POST /api/content/[id]/publish]
• Server Enforces: post.status === "approved" (400 if not)
• Server Enforces: requested channels match approved selection
[Website CMS API]
/api/sandbox/cms/posts
[LinkedIn Posts API]
/api/sandbox/linkedin
[Newsletter Dispatch]
/api/sandbox/newsletter
▼ (Status: "published" / "partially_published")
[Neon DB: PublishingLog & SandboxPublication]
[Live Previews: /sandbox/news/*, /sandbox/linkedin/*]

## 2. Tools & Technology Stack

| Layer / Capability | Technology Chosen | Architectural Rationale |
|---|---|---|
| **Web Framework** | Next.js 16.4.2 (App Router, Turbopack) | Modern React 19 server components, unified API routing, zero-bundle static rendering, and instant edge compilation. |
| **Database & ORM** | Neon Serverless PostgreSQL + Prisma 7 | Managed branching PostgreSQL with connection pooling via `@prisma/adapter-pg`, guaranteeing acid transactions and strict schemas. |
| **AI Multimodal Engine** | Google Gemini 3.1 Flash / Flash-Lite | Native multimodal document parsing (PDF magic bytes, PNG, JPEG) and structured JSON schema enforcement at low latency. |
| **Workflow Automation** | n8n (Local Docker/npm & n8n Cloud) | Visual orchestration, external trigger scheduling, fail-closed HMAC authentication, and parameterization. |
| **Validation Engine** | Custom Functional TypeScript Validators | Strict mathematical bounds, RFC 4180 CSV compliance, ISO 4217 currency normalization, and magic-byte sniffers without bloated dependencies. |
| **Test Runner** | Node.js 22 Native Test Runner (`node:test`) | Zero-dependency, high-speed automated testing running 89 unit tests across 12 suites in under 1.2 seconds. |
| **Styling & Design System** | Tailwind CSS 4 + Lucide React | Utility-first responsive design, Apple Human Interface inspired feedback states, and dark mode support. |

## 3. Integrations

### 1. Google Gemini API Multimodal & Structured JSON

- **Multimodal Document Processing**: Converts uploaded raw file buffers into inline Base64 data with validated MIME types (`application/pdf`, `image/png`, `image/jpeg`).
- **System Prompts & Structured Response**: Enforces strict JSON Schema contracts for financial extraction (vendor, invoiceNumber, expenseDate, amount, currency, category, line items) and multi-channel content generation (LinkedIn post, newsletter briefing, CMS markdown).
- **Header-Only Authentication**: Transmits credentials strictly via the `x-goog-api-key` header, eliminating key leaks in query strings or server logs.

### 2. n8n Workflow Orchestration & Fail-Closed Webhooks

- **Fail-Closed Security Model**: Inbound webhooks evaluate the `x-webhook-secret` header against the server secret. If missing or mismatched, requests immediately terminate with HTTP 401/500 without

invoking AI or database nodes.

- **SQL Parameterization**: PostgreSQL nodes use query replacements ( `$1..$N` ), preventing SQL injection vulnerabilities.
- **Automatic Fallback**: Next.js API clients monitor n8n health. If n8n is offline (503) or unconfigured, the application falls back seamlessly to the internal direct TypeScript pipeline.

### 3. Neon PostgreSQL with Prisma 7 Adapter Pooler

- **Connection Pooling**: Utilizes `@prisma/adapter-pg` on top of standard `pg.Pool` to support high-concurrency serverless execution without pool exhaustion.
- **Audit Logging & Relational Integrity**: `Expense` , `ContentPost` , `PublishingLog` , and `SandboxPublication` models maintain relational foreign keys, timestamps, and indexes.

### 4. Multi-Channel Sandbox Simulation System

- **Live Local Inspection**: Provisions mock platform endpoints ( `POST /api/sandbox/cms/posts` , `POST /api/sandbox/linkedin/posts` , `POST /api/sandbox/newsletter/send` ) protected by Bearer token authentication.
- **Dedicated Preview Viewers**: Renders high-fidelity channel mockups (LinkedIn feed card, Substack-style newsletter, newsroom article) accessible at `/sandbox/news/[slug]` , `/sandbox/linkedin/[id]` , and `/sandbox/newsletter/[id]` .
- **Centralized Sandbox Hub**: Interactive overview page at `/sandbox` listing all mock publications with timestamps and direct preview links.

# 4. Limitations & Design Invariants

## A. Strict System Invariants

| Invariant Name | Rule & Enforcement | System Behavior |
|---|---|---|
| **Arithmetic Tolerance** | `subtotal + tax == total` within $0.02 | Records `ARITHMETIC_MISMATCH` issue if violated; sets status to `needs_review` without discarding data. |
| **Zero Cross-Currency Addition** | Strict ISO 4217 currency segregation | Financial reporting aggregates volumes strictly by currency; never sums disparate currencies into a single total. |
| **Two-Tier Duplicate Engine** | Tier 1: Vendor + Inv# + Amount + Curr<br>Tier 2: Vendor + Date + Amount + Curr | Flags `duplicate_suspected`; Tier 2 only fires when invoice number is absent from both incoming and existing records. |
| **Idempotency Fast-Path** | Unique `idempotencyKey` constraint | Replays persisted expense in ~270ms on duplicate submissions, bypassing AI calls to save cost and quota. |
| **Human Approval Guardrail** | Server-side rejection on unapproved publish | `POST /api/content/[id]/publish` returns HTTP 400 if `status !== 'approved'`. Client publish button remains disabled. |
| **Re-Approval on Edit** | Modifications invalidate prior approval | Any edit to copy or channel destinations resets post status to `in_review`, clears `approvedAt`, and blocks publishing until re-approved. |
| **Failed Post Re-Approval** | Editing a failed post resets approval | If publishing fails across channels, subsequent edits clear approval to prevent unreviewed content dispatch on retry. |
| **Honest Sandbox Transparency** | Clear simulated badges and statuses | Sandbox dispatches record status `"simulated"` in `PublishingLog`. UI displays **Demo Reference (Simulated)**. |

## B. Architectural Limitations & Scope Decisions

- **Sandbox Publishing vs. Live Credentials**: The assessment uses authenticated mock endpoints rather than production social media tokens (e.g. LinkedIn Marketing Developer Enterprise access), avoiding token expiration and credential exposure during review.
- **Constant-Time Passcode vs. Full OAuth/OIDC**: The human review guard uses constant-time header comparison (`x-reviewer-passcode`) for streamlined evaluation rather than requiring a multi-tenant OAuth/SAML server.

- **Single-Tenant Schema**: Designed as a focused, self-contained assessment demonstration rather than a multi-organization SaaS architecture.
- **Google Gemini API Rate Limits**: Subject to standard Gemini API quota bounds; resilient exponential backoff retry handles transient 429 rate spikes.

## 5. Instructions for Reproducing the Workflow

### Step 1: Prerequisites & Clone

```
# Requires Node.js 22+ and Git
git clone https://github.com/Ahmed-Abd-EL-Kareem/impact-circles-ai-automation.git
cd impact-circles-ai-automation
npm install
```

### Step 2: Environment Configuration

Copy the example environment file:

```
cp .env.example .env    # On Windows: copy .env.example .env
```

Configure required keys in `.env`:

```
DATABASE_URL="postgresql://user:password@ep-xxxx.region.aws.neon.tech/neondb?sslmode=require"
DIRECT_URL="postgresql://user:password@ep-xxxx.region.aws.neon.tech/neondb?sslmode=require"
GEMINI_API_KEY="your-google-gemini-api-key"
GEMINI_MODEL="gemini-flash-latest"
SANDBOX_BASE_URL="http://localhost:3000"
SANDBOX_API_KEY="sandbox_demo_secret_key"
```

### Step 3: Database Migration

```
npx prisma generate
npx prisma migrate deploy
```

### Step 4: Run Application (Built-in Pipeline)

By default, n8n variables remain commented out in `.env`, and the app runs out-of-the-box using its built-in direct pipeline:

```
npm run dev    # Open http://localhost:3000
```

## Step 5: (Optional) Running with n8n — Local or n8n Cloud

### Option A: Local n8n Instance

```
# Start n8n locally
npx n8n
# Or via Docker:
# docker run -it --rm --name n8n -p 5678:5678 -v ~/.n8n:/home/node/.n8n n8nio/n8n
```

Uncomment in `.env`:

```
N8N_BASE_URL="http://localhost:5678"
N8N_WEBHOOK_URL="http://localhost:5678/webhook/impact-circles-ai-automation"
N8N_CONTENT_WEBHOOK_URL="http://localhost:5678/webhook/impact-circles-content-publishing"
N8N_WEBHOOK_SECRET="your-webhook-shared-secret"
```

### Option B: n8n Cloud Workspace

Point `.env` to your cloud instance:

```
N8N_BASE_URL="https://your-instance.app.n8n.cloud"
N8N_WEBHOOK_URL="https://your-instance.app.n8n.cloud/webhook/impact-circles-ai-automation"
N8N_CONTENT_WEBHOOK_URL="https://your-instance.app.n8n.cloud/webhook/impact-circles-content-publishing"
N8N_WEBHOOK_SECRET="your-webhook-shared-secret"
```

### Importing & Activating Workflows (Local or Cloud)

1. Open n8n, navigate to **Workflows** → **Import from File**.
2. Import `n8n/workflows/finance-processing.json` and `n8n/workflows/content-publishing.json`.
3. Add a **Postgres Credential** pointing to your Neon database URL (Host, Database, User, Password, Port 5432, SSL: `require`).
4. Add a **Google Gemini API Credential** using your `GEMINI_API_KEY`.
5. Ensure `N8N_WEBHOOK_SECRET` is configured in n8n's environment variables to match `.env`.
6. Toggle both workflows to **Active**.

## Step 6: Automated Verification Suite (89 Tests)

```
# Run all 89 unit tests across 12 test suites (100% pass)
npm test

# Run TypeScript compilation and Turbopack production build
npx tsc --noEmit
npm run lint
npm run build

# Run automated integration tests
npm run test:direct        # Probes Neon DB + Gemini directly
npm run test:e2e           # Probes live n8n workflow on port 5678
npm run export:samples     # Exports real pipeline samples into sample-outputs/
npm run verify:e2e         # Executes automated verification script
```

## Step 7: 5-Minute Manual Verification Walkthrough

1. **Finance Ingestion**: Drag and drop `test-fixtures/fictional-invoice.pdf` on the Finance tab. Observe Gemini multimodal extraction, subtotal/tax verification, automatic approval, and ledger entry.
2. **Duplicate Protection**: Re-upload the same invoice. Observe immediate detection with status `duplicate_suspected` and candidate ID preservation.
3. **Missing Fields Handling**: Upload `test-fixtures/fictional-missing-fields-invoice.pdf`. Observe honest null parsing and status `needs_review`.
4. **Category Spend & RFC 4180 CSV Export**: View the category breakdown table with strict currency isolation. Click "Export CSV" to download the compliant financial record.
5. **Multi-Channel Content Generation**: Switch to the Content Agent tab. Enter a research topic and click "Generate Multi-Channel Content". Gemini produces LinkedIn, Newsletter, and Website formats simultaneously.
6. **Human Review & Re-Approval Invariant**: Open "Review & Approve". Edit copy or change target channels. Observe that status resets to `in_review`, clearing prior approvals.
7. **Publishing & Live Preview**: Approve content and click "Publish to Sandbox". Click the returned preview URLs to inspect live mockups on `/sandbox/news/[slug]`, `/sandbox/linkedin/[id]`, and `/sandbox/newsletter/[id]`.

> **Summary & Deliverable Verification**
>
> All 89 unit tests, Turbopack production build, and ESLint checks pass with 100% success rate. The repository is fully reproducible, fail-closed secure, and production ready.